

Reviewer Pack — Minimal Tropical Root Separation and ACC0 Lower Bounds

Entry 046406fb9e61 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 19:06:28 UTC

Minimal Tropical Root Separation and ACC0 Lower Bounds

Entry ID: 046406fb9e61 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 19:06:28 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): ACC⁰ Circuit Complexity

Statement:

[‘For every explicit function f in P , there exists a tropical curve C_f such that the minimal root separation of C_f is at least $\Omega(n^{1/3})$ if and only if f requires an ACC⁰ circuit of size at least n^2 .’, ‘The minimal root separation of a tropical curve C_f is defined as the minimum distance between any two points on C_f .’, ‘The minimal root separation of C_f is computable in polynomial time.’]

Rationale (proposer’s reasoning):

[‘This conjecture links tropical geometry, which has previously not been applied to ACC⁰ lower bounds, with circuit complexity. The invariant provides a potential bridge for proving lower bounds, as it is sensitive to the structure of functions that require larger circuits.’, ‘Tropical geometry offers a rich algebraic-geometric framework that can capture the complexity of computations, and its use here could expose new insights into ACC⁰.’, ‘If proven, this conjecture would provide a novel approach to proving ACC⁰ lower bounds for explicit functions.’]

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6f6e8ae96fb117ab

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all explicit functions in P , the computed minimal root separation of their corresponding tropical curve C_f is at least $\Omega(n^{1/3})$ AND the size of the smallest ACC⁰ circuit for each function is at least n^2 . The conjecture is falsified if either condition fails for a single function.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "ACC0 circuit complexity" - "minimal root separation tropical curve" AND "ACC0 lower bounds" - "polynomial time computability minimal root separation" AND "function f P ACC0 circuit size"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_polynomial(n):
        coefficients = [random.randint(1, 10) for _ in range(n + 1)]
        return coefficients

    def evaluate_polynomial(poly, x):
        result = 0
        for i, coeff in enumerate(poly):
            result += coeff * (x ** i)
        return result

    def find_roots(poly):
        n = len(poly) - 1
        if n == 0:
            return []
        elif n == 1:
            return [-poly[0] / poly[1]]

        roots = []
        for i in range(n + 1):
            x = random.uniform(-10, 10)
            while abs(evaluate_polynomial(poly, x)) > 1e-6:
                x += random.uniform(-0.1, 0.1)
            roots.append(x)
```

```

    return roots

def minimal_root_separation(roots):
    if len(roots) < 2:
        return float('inf')
    return min(abs(roots[i] - roots[j]) for i in range(len(roots)) for j in range(i + 1, len(roots)))

def acc0_circuit_size(poly):
    n = len(poly) - 1
    if n == 0:
        return 1
    elif n == 1:
        return 2

    size = n * (n + 1) // 2
    for i in range(n + 1):
        size += abs(poly[i])

    return size

def is_in_p(poly):
    # Placeholder function to check if a polynomial is in P
    return True

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    poly = generate_polynomial(n)
    if not is_in_p(poly):
        continue

    roots = find_roots(poly)
    min_separation = minimal_root_separation(roots)
    circuit_size = acc0_circuit_size(poly)

    results.append({
        "n": n,
        "min_separation": min_separation,
        "circuit_size": circuit_size
    })

if not results:
    return {
        "metric_name": "minimal_root_separation",
        "metric_value": 0.0,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "No valid polynomials found"
    }

min_separations = [result["min_separation"] for result in results]
circuit_sizes = [result["circuit_size"] for result in results]

mean_min_separation = sum(min_separations) / len(min_separations)
std_min_separation = math.sqrt(sum((x - mean_min_separation) ** 2 for x in min_separations) / len(min_separations))

```

```

mean_circuit_size = sum(circuit_sizes) / len(circuit_sizes)

conjecture_holds = all(ms >= n**(1/3) and cs >= n**2 for ms, cs, n in zip(min_separations, circuit_sizes, n_values))
counterexample = "" if conjecture_holds else "None found"

return {
    "metric_name": "minimal_root_separation",
    "metric_value": mean_min_separation,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"None found\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, making it impossible to verify the conjecture's conditions. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14873
2	preregistration	ollama_remote	glm4:latest	0	0	11663
3	novelty	ollama_remote	glm4:latest	0	0	8337
4	novelty	ollama_remote	glm4:latest	0	0	9740
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21070
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10333
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11664
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13013
9	judge	ollama_remote	glm4:latest	0	0	12078

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 112772 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/046406fb9e61.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/046406fb9e61.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/046406fb9e61.tar.gz` (if generated)